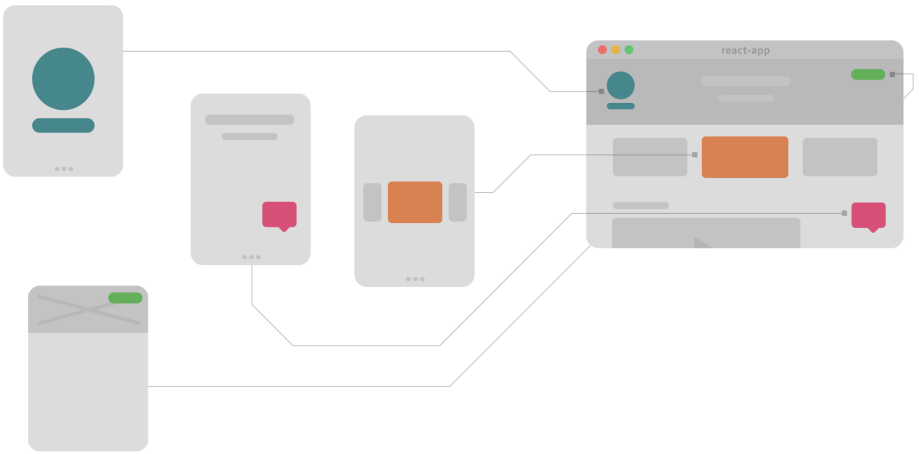
9. Components in React

▼ What is a Component in React?

- React follows a component-based architecture.
- Components are the building blocks of Applications.
- Heart of React's application is the Component.
- Components are like containers or bags.
- We write functions that return HTML elements.
- When creating the components, we must use Upper case letters.
- **For Example** → If we were building the UI of React website using React.js we can break its UI smaller parts.



▼ Types of Components

- There are two types of Components.
  1. Functional-Based Components
  2. Class-Based Components

▼ Naming Conventions for React Components

▼ 1. Use PascalCase for Component Names

- **PascalCase** means the first letter of **each word is capitalized**, with no spaces or underscores.
- **Why?**
  - React treats **components starting with uppercase letters** as **custom components**, not HTML tags.
  - ✅**Correct:**

```
function MyComponent() {
  return <h1>Hello</h1>;
}

<MyComponent />
```

  - ❌**Incorrect:**

```
function mycomponent() {
  return <h1>Hello</h1>;
}

<mycomponent />
```

// ❌ React will treat this as an unknown HTM
L element.

▼ 2. File Names Should Match Component Names

- This improves clarity and makes file searching easier.
- Example:
- Component: `UserProfile`
- File Name: `UserProfile.jsx`
- ✅ This helps when importing:

```
import UserProfile from './UserProfile';
```

▼ 3. Use Descriptive Names

- Component names should reflect **what the component does or represents**.
- ❌**Bad:**

```
function PC() {}
```

- ✅**Good:**

```
function ProductCard() {}
```

▼ 4. Avoid Abbreviations or Shortcuts

- Use full, meaningful names rather than cryptic shortcuts.
- ❌**Bad:**

```
function Btn() {}
```

- ✅**Good:**

```
function Button() {}
```

▼ Why These Conventions Matter

- React uses capital letters to differentiate between HTML tags and components.
- Following naming rules improves **collaboration**, **readability**, and **debugging**.

▼ 1. Functional-Based Component

- Functional-Based Components are the simple JavaScript functions that may or may not be accept as props and return JSX (UI Code).

- They describe what you want to display on the screen.
- The functional-based components does not have any state object internally.
- But we can make functional components as stateful components by using **Hooks** in React js.
- Nowadays, `React 16.8` onwards they introduced a concept called **hooks**.
- Nowadays, we don't use class-based components because it is complex compared to functional-based components.

▼ How to create function-based components

- We can create a function-based component in React using both the:
  1. By using `function` keyword
  2. By using `Arrow function` syntax

▼ 1. Using `function` Keyword (Traditional Syntax)

- **Syntax:**

```
function ComponentName(props) {
  return (
    <div>
      <h2>Hello</h2>
    </div>
  );
}
```

- **Example:**

```
function Welcome(props) {
  return <h1>Welcome!</h1>;
}

export default Welcome;
```

▼ 2. Using `Arrow` Function (Modern Syntax)

- **Syntax:**

```
const ComponentName = (props) => {
  return (
    <div>
      <h2>Hello</h2>
    </div>
  );
}
```

- **Example:**

```
const Welcome = (props) => {
  return <h1>Welcome!</h1>;
};

export default Welcome;
```

▼ 2. Class-Based Component

- Class-based components in React are JavaScript classes that extend from `React.Component` or `React.PureComponent`. (React is the base class where we have a property called as Component)
- They were the primary way to create stateful components before the introduction of hooks in React 16.8.
- They are defined using ES6 class syntax.

> 💡 Before **React Hooks** (introduced in v16.8), class components were the only way to handle **state** and **lifecycle methods** in React.

▼ Basic Syntax of Class Component

- **Syntax:**

```
import React from "react";

class App extends React.Component {
  render() {
```

```
    return <h1>This is class based component</h1>;
  }
}

export default App;
```

▼ `state` object in Class-Based Component

- A React class-based component has a built-in state object.
- The state object is where you store property values that belong to the component.
- When the state changes the component re-renders.
- The state object in a class-based component only accepts two types of data, `either null or an object`.
- In class-based components, we can use the constructor method to initialize the state object.
- State is the data/information isolated to a component.
- **For Example:**

▼ How do they work?

1. **constructor(props)**
   - The `constructor` is a special **method** in ES6 class used to **initialize the component**.
   - It runs **once** when the component is created.
   - If you're using state in a class component, you **must call** `super(props)` inside the constructor before using `this`.
   - **Example:**

```
constructor(props) {
  super(props);
  this.state = {
    count: 0
```

```
  };
}
```

2. **super(props)**
   - `super()` is used to call the **constructor of the parent class** (in our case, `React.Component`).
   - If you're using `constructor(props)`, you **must call** `super(props)` before accessing `this` or using `this.props`.
   - **Why?**
     - Because React components extend the base `Component` class, so you need to initialize that base class first.
3. `this` **keyword**
   - `this` refers to the **current instance** of the component.
   - You use `this` to access:
     - `this.props` → Props passed to the component
     - `this.state` → Component's local state
     - `this.setState()` → Update the state
     - `this.methodName` → Any method defined inside the class
4. **render()**
   - `render()` is a **required method** in every class component.
   - It returns **JSX** — the UI output that should be shown on the screen.
   - Every time the state or props change, React calls `render()` again to update the UI.

▼ Difference Between Function and Class-Based Component

- Difference:

Differences between Function Components and Class Components		
	Function Components	**Class Components**
Definition	JavaScript function that returns a JSX element	JavaScript class that extends React.Component and has a render() method that returns a JSX element
State	Uses React Hooks to manage state	Has built-in state management using this.state
Lifecycle Methods	Uses React Hooks to handle lifecycle events like `useState`, `useEffect`, `useMemo`.	Uses class lifecycle methods like `componentDidMount`, `componentDidUpdate`, and `componentWillUnmount`
Props	Props are passed as a parameter to the function	Props are accessed through this.props
Render	The component is defined within the function body and returned as a JSX element	The render() method is required and returns a JSX element
Code Clarity	Simple and concise	More verbose and harder to read
Performance	Sets you up to follow best practices	Easier to cause performance issues

---

▼ React Hooks

- The `React 16.8` release in `2019` changed everything.
- It introduced **React hooks**.
- React hooks are functions that let functional components manage state, handle lifecycle events, and access other React features.
- Before hooks, you could only do that using class components.